**SOKOINE UNIVERSITY OF AGRICULTURE**

Department of Informatics and Information Technology (DIIT)

# BIT 111: MICROCOMPUTERS AND APPLICATIONS

## TOPIC 4: FUNCTIONS AND TYPES OF OPERATING SYSTEMS — LECTURE COMPENDIUM

Audience: First-year undergraduate students
Approach: Introductory concepts, guided examples, and discipline-relevant applications
Topic overview: Operating-system roles, interfaces, file management, and common commands

# Topic 4: Lecture Outline & Learning Roadmap

## Operating Systems, Process Management & Commands (Coverage: 4.1, 4.2, 4.3)

• • Section 4.1: OS Evolution, Core Architecture & Subsystem Categories

• - The OS as Extended Machine & Resource Arbiter; Kernel models (Monolithic, Microkernel, Hybrid)

• - Dual-mode operation (Ring 0 vs Ring 3); System calls and interrupt handling

• - Process management: PCB, 5-state lifecycle, CPU scheduling algorithms (FCFS, SJF, SRTF, RR)

• - Memory management: Logical vs physical addressing, Paging, TLB, Virtual memory

• - File systems: FAT32, NTFS, ext4; Allocation structures and Journaling

• • Section 4.2: Disk Operating System (MS-DOS) Architecture & Commands

• - DOS architecture: `IO.SYS`, `MSDOS.SYS`, `COMMAND.COM`

• - Internal commands (`DIR`, `COPY`, `DEL`, `MD`, `CD`, `TYPE`) vs External (`FORMAT`, `CHKDSK`, `XCOPY`)

• - Batch scripting (`.BAT`): Variables, parameters `%1-%9`, conditionals `IF`, loops `FOR`, redirection

• • Section 4.3: Modern GUI Operating Systems (Windows & Linux) and Administration

• - Windows architecture: Registry, Task Manager, Event Viewer, PowerShell cmdlets

• - Linux architecture: Filesystem Hierarchy Standard (FHS), shell pipelines, process controls (`ps`, `top`, `kill`)

**PEDAGOGICAL OBJECTIVE**

Understand how the operating system bridges the gap between raw microprocessor silicon and user applications through process abstraction, resource scheduling, and command interfaces.

Study prompt: Explain this idea in your own words and connect it to an example from your discipline.

# Topic 4: Intended Learning Outcomes (ILOs)

## Core Operating Systems Competencies for SUA Undergraduates

• • Explain the dual roles of an operating system: Extended Machine (Abstraction) and Resource Arbiter.

• • Contrast Monolithic kernels, Microkernels, and Hybrid kernels in performance and fault isolation.

• • Explain process states and compare operating-system scheduling approaches conceptually.

• • Detail virtual memory address translation using the MMU, Page Tables, and TLB.

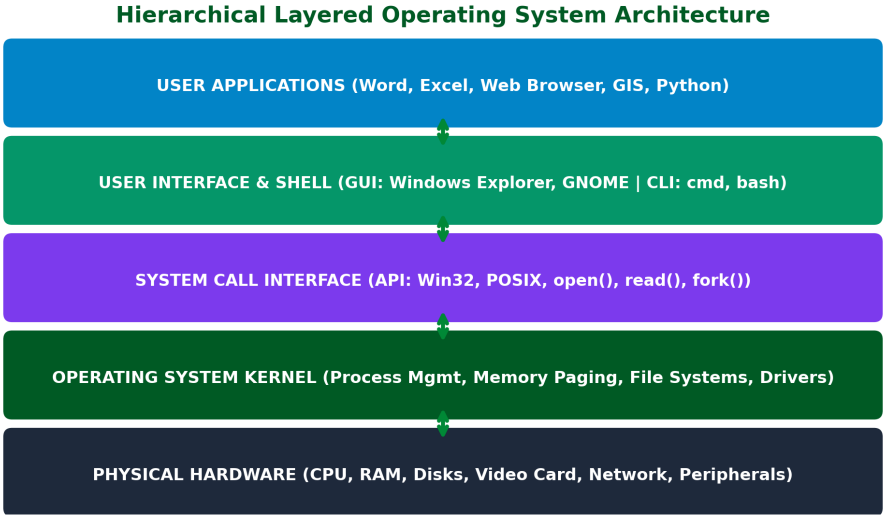
• • Distinguish internal MS-DOS commands from external disk executables and write automated batch scripts.

• • Evaluate file systems (FAT32, NTFS, ext4) regarding journaling, volume limits, and permissions.

• • Execute administrative system commands in Windows (PowerShell) and Linux (Bash) environments.

Study prompt: Explain this idea in your own words and connect it to an example from your discipline.

# Operating System Architectural Hierarchy




*The multi-tiered OS architectural model bridging hardware, kernel, system calls, and user applications.*

• • Hardware Layer: Physical CPU, memory, buses, timers, and peripheral controllers.

• • Kernel Mode (Ring 0): Unrestricted hardware access executing process scheduling, memory allocation, and device drivers.

• • System Call Interface: Protected software interrupt gate (e.g., `sys_enter`, `syscall`, `int 0x80`).

• • User Mode (Ring 3): Sandboxed execution of user applications, shell CLI, and GUI desktop environments without direct hardware access.

Study prompt: Explain this idea in your own words and connect it to an example from your discipline.

# OS Kernel Architectural Paradigms

## Monolithic Kernels vs Microkernels vs Hybrid Models

• • Monolithic Kernels (e.g., Linux, traditional UNIX):

• - All core OS services (scheduler, memory manager, file systems, device drivers, network stack) run together in Ring 0.

• - Advantage: Ultra-fast performance; direct function calls between subsystems.

• - Disadvantage: A single buggy device driver can crash the entire operating system (Kernel Panic).

• • Microkernels (e.g., Mach, QNX, seL4):

• - Ring 0 stripped down to bare minimum: IPC (Inter-Process Communication), basic scheduling, low-level memory.

• - Device drivers, file systems, and network stacks execute in User Space (Ring 3) as modular servers.

• - Advantage: Extreme fault tolerance and security; crash of a driver does not crash the kernel.

• • Hybrid Kernels (e.g., Windows NT kernel, Apple macOS XNU):

• - Combines microkernel modularity with monolithic performance by keeping select drivers in kernel space.

Study prompt: Explain this idea in your own words and connect it to an example from your discipline.

# Dual-Mode Operation & Hardware Protection

## Preventing Errant User Programs from Crashing Microcomputers

• • The Dual-Mode Protection Mechanism (enforced by CPU hardware status bit):

• - 1. User Mode (Ring 3): Application execution mode; privileged CPU instructions (e.g., disabling interrupts, direct I/O port writes) are strictly forbidden.

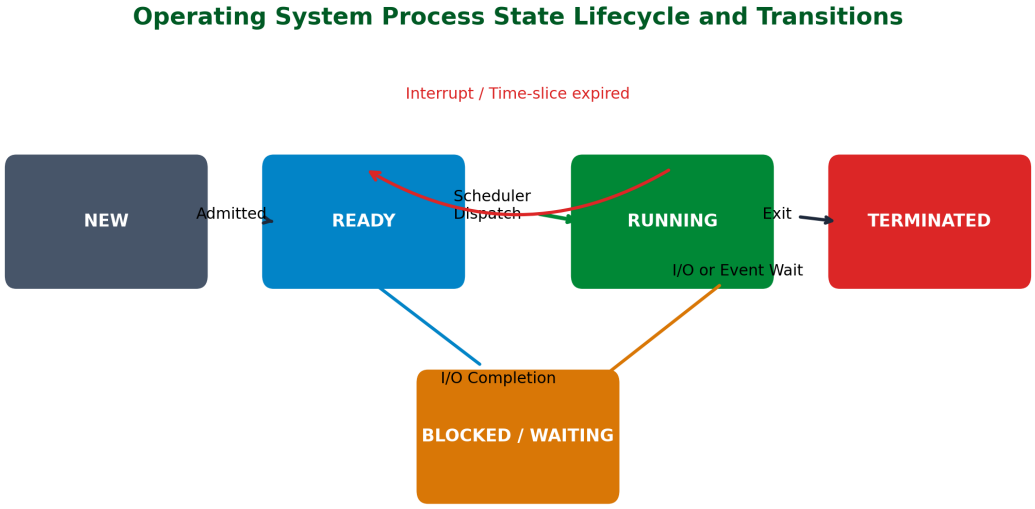
• - 2. Kernel / Supervisor Mode (Ring 0): Full instruction set enabled with direct hardware manipulation.

• • System Calls (Syscalls): The sanctioned software API for user applications to request kernel services (e.g., `read()`, `write()`, `fork()`, `open()`).

• • Mechanism: Application sets up arguments in registers, triggers a software trap/interrupt; CPU switches from Ring 3 to Ring 0, kernel validates parameters, executes task, and returns control switching back to Ring 3.

Study prompt: Explain this idea in your own words and connect it to an example from your discipline.

# The Five-State Process Lifecycle




*The classical 5-state process lifecycle illustrating CPU scheduling and I/O wait transitions.*

- • New: Process being created and its Process Control Block (PCB) allocated.
- • Ready: Process residing in main memory, waiting for the OS CPU scheduler to dispatch it.
- • Running: Instructions currently executing on the physical CPU core.
- • Waiting (Blocked): Process suspended waiting for an external event (disk I/O, keyboard input, network packet).
- • Terminated: Process completed execution; resources deallocated by the kernel.

Study prompt: Explain this idea in your own words and connect it to an example from your discipline.

# Process Management & The Process Control Block (PCB)

## The Kernel's Data Structure Representing a Living Process

• • Process Definition: A program in execution; an active entity with a program counter, memory segments, and assigned OS resources.

• • Process Control Block (PCB) Data Fields:

• - Process Identifier (PID): Unique integer assigned by kernel.

• - Process State: New, Ready, Running, Waiting, or Terminated.

• - Program Counter (PC): Memory address of next instruction to execute.

• - CPU Registers: Saved accumulator, index, stack pointer, and flag registers.

• - CPU Scheduling Info: Process priority and queue pointers.

• - Memory Management Info: Base/limit registers, page table pointers.

• - Accounting & I/O Status: CPU time used, open file descriptors, allocated I/O devices.

• • Context Switching: The CPU state save and restore cycle when preempting one process for another (pure computational overhead).

Study prompt: Explain this idea in your own words and connect it to an example from your discipline.

# CPU Scheduling Approaches

## How the operating system shares processor time

• • The operating system chooses which ready process can use the processor.

• • First-Come, First-Served runs processes in the order they arrive.

• • Shortest Job First gives priority to work expected to finish sooner.

• • Round Robin gives each ready process a brief turn in sequence and is useful for interactive systems.

• • Scheduling approaches make different trade-offs between fairness, responsiveness, and simplicity.

Study prompt: Explain this idea in your own words and connect it to an example from your discipline.

# Comparing Scheduling Approaches

## A conceptual example from a shared computer lab

- • In a shared laboratory, several users may open applications at the same time.
- • First-Come, First-Served is simple to understand, but a long task can delay later tasks.
- • Round Robin gives each ready process a turn, helping interactive applications remain responsive.
- • A scheduling policy should balance responsiveness and fair access.
- • Compare the approaches by describing their behavior; no timing calculations are required.

Study prompt: Explain this idea in your own words and connect it to an example from your discipline.

# Concurrency, Race Conditions, and Deadlocks

## Multi-Thread Synchronization and The Coffman Conditions

• • Race Condition: A situation where multiple concurrent threads access shared data, and the final outcome depends on the non-deterministic timing of thread execution.

• • Critical Section: Code segment accessing shared resources that must be executed with Mutual Exclusion (mutex locks, semaphores).

• • Deadlock: A permanent stalling state where two or more processes are unable to proceed because each holds a resource the other needs.

• • The Four Coffman Conditions (Must all hold simultaneously for deadlock):

• 1. Mutual Exclusion: At least one resource must be held in non-shareable mode.

• 2. Hold and Wait: A process holds resources while requesting additional ones.

• 3. No Preemption: Resources cannot be forcibly seized from a holding process.

• 4. Circular Wait: A closed chain of processes exists where P0 waits for P1, and Pn waits for P0.

Study prompt: Explain this idea in your own words and connect it to an example from your discipline.

# Memory Management: Paging & Virtual Memory

## Translating Virtual Addresses to Physical DRAM Frames

• • Logical (Virtual) vs Physical Address Space: Applications operate in contiguous virtual memory isolated from actual fragmented physical RAM.

• • Paging Mechanism: Divides virtual memory into fixed-size blocks called Pages (typically 4 KiB); physical memory divided into Frames of identical size.

• • Memory Management Unit (MMU): Hardware on the CPU die translating virtual page numbers (VPN) to physical frame numbers (PFN) via Page Tables.

• • Translation Lookaside Buffer (TLB): High-speed on-die associative cache buffering recent address translations, eliminating memory lookup overhead.

• • Demand Paging & Page Faults: Pages loaded into DRAM only when accessed; referencing an unmapped page triggers a Page Fault interrupt, causing the OS to fetch the page from secondary storage (swap space).

Study prompt: Explain this idea in your own words and connect it to an example from your discipline.

## Enterprise File Systems Comparison

| Feature / Attribute | FAT32 | exFAT | NTFS | ext4 (Linux) |
|---|---|---|---|---|
| Max File Size | 4 GiB | 16 EiB | 16 TiB - 8 PiB | 16 TiB |
| Max Volume Size | 2 TiB (32-bit) | 128 PiB | 8 PiB | 1 EiB |
| Journaling Support | No (Corruption prone) | No | Yes (USN Journal) | Yes (JBD2 Journal) |
| Access Control (ACLs) | None (Universal read) | None | Full NTFS Permissions | Full POSIX Permissions |
| Native Encryption | None | None | EFS (Encrypting FS) | fscrypt (Native) |
| Target Ecosystem | Legacy USB Drives | Cross-Platform Flash | Windows OS & Servers | Linux OS & Servers |

Study prompt: Explain this idea in your own words and connect it to an example from your discipline.

# MS-DOS Architecture: Three Core Files

## The 16-Bit Real-Mode Disk Operating System

• • Historical Significance: 16-bit single-user, single-tasking command-line OS created by Tim Paterson (86-DOS) and licensed by Microsoft as MS-DOS / PC-DOS (1981).

• • The Three Fundamental System Files:

• - 1. `IO.SYS`: Low-level hardware interface containing basic device drivers (screen, keyboard, serial, floppy); interfaces directly with ROM BIOS.

• - 2. `MSDOS.SYS`: The DOS operating system kernel; manages the file system, memory allocation, directory structures, and system call interrupts (`INT 21h`).

• - 3. `COMMAND.COM`: The default command-line command interpreter (shell); parses user commands, executes batch files, and houses all internal commands.

• • Boot Sequence: ROM BIOS POST → MBR → Boot Sector → Loads `IO.SYS` → Loads `MSDOS.SYS` → Parses `CONFIG.SYS` → Executes `COMMAND.COM` → Runs `AUTOEXEC.BAT`.

Study prompt: Explain this idea in your own words and connect it to an example from your discipline.

# MS-DOS Internal Commands

## Resident in RAM: Handled Directly by COMMAND.COM

• • Definition: Built directly into `COMMAND.COM`; always resident in main memory, executing instantly without requiring disk access.

• • Essential Internal Directory & File Commands:

• - `DIR`: Lists files, subdirectories, file sizes, and timestamps in current directory.

• - `MD` or `MKDIR`: Creates a new subdirectory (e.g., `MD D:\SUA_DATA`).

• - `CD` or `CHDIR`: Changes the current working directory (e.g., `CD \ACADEMIC\BIT111`).

• - `RD` or `RMDIR`: Removes an empty directory.

• - `COPY`: Copies one or more files to another destination (e.g., `COPY notes.txt E:\`).

• - `DEL` or `ERASE`: Deletes specified files from the disk.

• - `REN`: Renames files or directories without moving them.

• - `TYPE`: Displays contents of a text file on the screen.

• - `CLS`: Clears the display screen; `VER`: Displays operating system version.

Study prompt: Explain this idea in your own words and connect it to an example from your discipline.

# MS-DOS External Commands

## Disk-Resident Executables: Stored as .COM, .EXE, or .BAT Files

• • Definition: Separate executable files stored on secondary storage; loaded into RAM only when invoked by the user.

• • Essential External Disk & File Management Commands:

• - `FORMAT.COM`: Formats a disk volume with a file system, creating the boot sector, FAT tables, and root directory.

• - `CHKDSK.EXE`: Analyzes disk volume integrity, checks for lost allocation clusters, and reports free/bad sectors.

• - `XCOPY.EXE`: Advanced file/directory copy utility supporting recursive directory tree copying (`/S /E /Y /V`).

• - `ROBOCOPY.EXE` (Modern Windows): Robust file copying utility with multi-threading, restartable mode, and mirror synchronization.

• - `ATTRIB.EXE`: Views or modifies file attributes: Read-only (`+R`), Hidden (`+H`), System (`+S`), Archive (`+A`).

• - `TREE.COM`: Graphically displays directory path hierarchy of a drive.

Study prompt: Explain this idea in your own words and connect it to an example from your discipline.

# MS-DOS Batch Scripting & Automation

## Automating Multi-Step Operations via .BAT Files

• • Batch File: Plain text file containing sequential DOS commands executed by `COMMAND.COM` or Windows `cmd.exe`.

• • Key Syntax Elements:

• - `@ECHO OFF`: Suppresses command echoing to clean up terminal output.

• - Command-line arguments: Positional variables `%1`, `%2`, through `%9`.

• - Conditionals: `IF EXIST filename (command) ELSE (command)`.

• - Flow Control: `GOTO label` and `:label` definitions.

• - Iteration: `FOR %%F IN (*.DAT) DO (command)`.

• - Redirection Operators: Output redirect `>` (overwrite), `>>` (append), Input redirect `<`, Pipe `|`.

Study prompt: Explain this idea in your own words and connect it to an example from your discipline.

# Modern GUI Operating Systems: The WIMP Paradigm

## Windows, Icons, Menus, and Pointer Interfaces

• • Evolution from CLI to GUI: Douglas Engelbart (NLS, 1968) → Xerox PARC (Alto, 1973) → Apple Macintosh (1984) → Windows 95.

• • WIMP Elements:

• - Windows: Rectangular screen viewports isolating running application contexts.

• - Icons: Small graphical pictograms representing files, programs, and system entities.

• - Menus: Drop-down or context-sensitive lists of executable options.

• - Pointer: Mouse/trackpad cursor translating physical 2D hand movement into on-screen spatial targeting.

• • Modern Desktop Environments: Windows Shell / Fluent Design; Linux GNOME, KDE Plasma, XFCE.

Study prompt: Explain this idea in your own words and connect it to an example from your discipline.

# Windows System Management Utilities

## Core MMC Consoles and Administrative Tools

• • Task Manager (`taskmgr`): Inspects running processes, CPU/RAM/Disk utilization, startup programs, and thread status.

• • Device Manager (`devmgmt.msc`): Hardware abstraction console displaying driver versions, IRQ assignments, and hardware device conflicts.

• • Disk Management (`diskmgmt.msc`): Initializes MBR/GPT partition tables, shrinks/extends volumes, formats NTFS/exFAT drives.

• • Event Viewer (`eventvwr.msc`): Centralized diagnostic log repository recording Application, Security, Setup, and System error logs.

• • Services MMC (`services.msc`): Controls background daemon processes (Start, Stop, Restart, Startup Type: Auto/Manual/Disabled).

• • Windows Registry (`regedit`): Hierarchical database storing OS configuration (`HKEY_LOCAL_MACHINE`, `HKEY_CURRENT_USER`).

Study prompt: Explain this idea in your own words and connect it to an example from your discipline.

# Modern Windows Administration: PowerShell

## Object-Oriented Automation vs Traditional Text Shells

• • Paradigm Shift: Unlike traditional text-based shells (DOS/Bash), PowerShell passes structured .NET objects across the pipeline, not raw character streams.

• • Verb-Noun Syntax: Consistent command naming pattern (`Get-Service`, `Stop-Process`, `Set-ExecutionPolicy`).

• • Essential Administrative Cmdlets:

• - `Get-Process | Where-Object WorkingSet -gt 500MB | Stop-Process`: Kills processes consuming >500 MB RAM.

• - `Get-Service -Name Spooler | Restart-Service`: Restarts print spooler daemon.

• - `Get-EventLog -LogName System -EntryType Error -Newest 10`: Extracts latest 10 critical system errors.

• - `Test-NetConnection -ComputerName www.sua.ac.tz -Port 443`: Verifies TCP socket reachability.

Study prompt: Explain this idea in your own words and connect it to an example from your discipline.

# Linux Architecture & Filesystem Hierarchy Standard

## The Structure of the Enterprise Open-Source OS

• • Philosophy: 'Everything is a file' — storage devices, serial ports, and processes are exposed via virtual filesystems.

• • FHS Directory Hierarchy:

• - `/bin` & `/sbin`: Essential user and system administration binary executables.

• - `/etc`: System-wide configuration files (e.g., `/etc/passwd`, `/etc/fstab`, `/etc/network`).

• - `/home`: User personal directories and application data.

• - `/var`: Variable data files (log files in `/var/log`, mail spools, web root `/var/www`).

• - `/dev`: Special device nodes representing hardware (`/dev/sda`, `/dev/nvme0n1`).

• - `/proc` & `/sys`: Virtual pseudo-filesystems exposing live kernel state and hardware metrics.

Study prompt: Explain this idea in your own words and connect it to an example from your discipline.

# Linux Terminal Diagnostics & Process Control

## Inspecting System Performance and Managing Daemons

• • Process Inspection: `ps aux` (BSD syntax), `ps -ef` (System V syntax), `top`, `htop` (interactive ncurses monitor).

• • Signal Transmission: `kill -15 PID` (SIGTERM - polite termination request allowing cleanup) vs `kill -9 PID` (SIGKILL - immediate uninterceptable kernel abort).

• • Process Scheduling Priority: `nice -n 10 command` (launches process with lower CPU priority); `renice -n -5 -p PID` (boosts running process priority).

• • Service Management via systemd: `systemctl status service_name`, `systemctl start/stop/restart`, `systemctl enable` (auto-start on boot).

• • Storage Telemetry: `df -h` (filesystem disk space usage), `du -sh /path` (directory space consumption).

Study prompt: Explain this idea in your own words and connect it to an example from your discipline.

# Real-Time Operating Systems (RTOS) in Agriculture

## Deterministic Guarantees vs Best-Effort Desktop Systems

• • RTOS Definition: An operating system designed to execute processes within strictly bounded, deterministic time constraints (deadlines).

• • Categories:

• - Hard Real-Time: Missing a single deadline is considered total system failure (e.g., autonomous tractor obstacle avoidance, airbag deployment).

• - Soft Real-Time: Missing a deadline degrades quality of service but does not cause catastrophic system failure (e.g., video streaming telemetry).

• • Why GPOS (Windows/Ubuntu) Fails at Real-Time: Non-deterministic kernel preemption latency, complex virtual memory page faults, and background garbage collection.

• • Agricultural Harvester Case Study: High-speed optical grain sorters use RTOS (e.g., FreeRTOS, VxWorks) to actuate pneumatic air-ejector valves in < 2 ms when rejecting discolored grains.

Study prompt: Explain this idea in your own words and connect it to an example from your discipline.

# OS Boot Architecture: Firmware to Init / Systemd (PID 1)

## The Step-by-Step Transition from Silicon to Multi-User Shell

• • Boot Sequence Stages:

• 1. POST & Firmware: UEFI initializes hardware, verifies CPU/RAM, executes Secure Boot validation.

• 2. Bootloader: UEFI executes EFI bootloader (GRUB2 on Linux / Windows Boot Manager `bootmgr.efi`).

• 3. Kernel Initialization: Kernel decompressed into RAM, sets up page tables, MMU, interrupts, device drivers.

• 4. Root Filesystem Mount: Mounts initial RAM disk (`initramfs`) to load storage drivers, then mounts real root `/`.

• 5. Init / Systemd (PID 1): The ancestor process of all user-space daemons, launching target services (networking, SSH, display manager).

• • Windows Equivalent: `ntoskrnl.exe` initialized → launches Session Manager Subsystem (`smss.exe`) → launches `csrss.exe` and `wininit.exe`.

Study prompt: Explain this idea in your own words and connect it to an example from your discipline.

# Page Replacement Algorithms: FIFO to LRU and Clock

## Managing Limited Physical Memory Under Heavy Workloads

• • The Page Replacement Problem: When a page fault occurs and all DRAM frames are occupied, the OS must evict a victim page to secondary storage.

• • 1. First-In, First-Out (FIFO): Evicts the oldest loaded page; simple queue; suffers from Belady's Anomaly (allocating more physical frames can paradoxically increase page faults).

• • 2. Optimal Page Replacement (OPT / MIN): Evicts the page that will not be used for the longest future duration; theoretically optimal but impossible to implement in practice (requires future knowledge).

• • 3. Least Recently Used (LRU): Evicts the page unused for the longest past time (approximates OPT based on temporal locality); high overhead to track timestamps.

• • 4. Clock Algorithm (Second-Chance): Practical approximation of LRU using a single reference bit per frame scanned circularly by a clock pointer.

Study prompt: Explain this idea in your own words and connect it to an example from your discipline.

# Virtual Memory Thrashing & The Working Set Model

## Understanding System Freezes Caused by Paging Collapse

• • Thrashing Definition: A state where the CPU spends more time swapping pages between RAM and disk than executing real user instructions.

• • The Collapse Mechanism: If the sum of working sets of all running processes exceeds total physical RAM, page fault frequency skyrockets → CPU utilization plummets to near zero → OS scheduler misinterprets low CPU usage and launches more processes, worsening the collapse.

• • Working Set Model (Peter Denning):

• - Working Set W(t, Δ): The set of pages referenced by a process in the most recent time window Δ.

• - Operating System Policy: The OS must ensure that each process has enough frames allocated to hold its working set. If total demand exceeds physical memory, the OS suspends (swaps out) entire processes to restore equilibrium.

Study prompt: Explain this idea in your own words and connect it to an example from your discipline.

# File Allocation Architecture: Inodes & Extents

## Contiguous, Linked List, and Indexed File Block Mapping

• • 1. Contiguous Allocation: File blocks stored in sequential disk sectors; fast sequential access; severe external fragmentation; difficult file expansion.

• • 2. Linked List Allocation: Each disk block contains a pointer to the next block; eliminates external fragmentation; slow random access; pointer corruption destroys remaining file.

• • 3. Indexed Allocation (UNIX Inode Architecture):

• - Inode: Metadata structure containing file size, permissions, timestamps, and an array of block pointers.

• - Direct Pointers (typically 12): Point directly to data blocks (fast access for small files).

• - Indirect Pointers: Single, Double, and Triple indirect pointers pointing to pointer tables, allowing files to scale to terabytes.

• • Modern Extent-Based Allocation: Extents (start block + length) used in ext4 and NTFS to reduce pointer overhead for large contiguous files.

Study prompt: Explain this idea in your own words and connect it to an example from your discipline.

# Hypervisors & Virtualization: Type-1 vs Type-2

## Hardware Abstraction and Multi-Tenant Server Consolidation

• • Virtual Machine (VM): Complete software emulation of an entire physical computer, including virtual CPU, virtual BIOS/UEFI, virtual RAM, and virtual storage.

• • Type-1 Hypervisor (Bare-Metal):

• - Installs and runs directly on bare physical server hardware (e.g., VMware ESXi, Proxmox VE, Microsoft Hyper-V Server, KVM).

• - Superior performance, low virtualization latency, high security isolation; the enterprise standard for university cloud datacenters.

• • Type-2 Hypervisor (Hosted):

• - Runs as an application inside an existing host operating system (e.g., Oracle VirtualBox, VMware Workstation).

• - Dependent on host OS stability; used primarily for student labs, testing, and cross-platform software development.

Study prompt: Explain this idea in your own words and connect it to an example from your discipline.

# Containerization: Linux Namespaces & Docker vs VMs

## Operating System-Level Virtualization and Microservices

• • Paradigm Difference: While Virtual Machines virtualize the underlying physical hardware, Containers virtualize the operating system kernel.

• • Linux Kernel Foundations of Containers:

• - Namespaces: Isolate process visibility across PID (process IDs), NET (network interfaces/ports), MNT (filesystem mounts), and IPC.

• - Control Groups (cgroups): Restrict and measure resource consumption (limiting container CPU cores, memory limits, I/O bandwidth).

• • Container Advantages over VMs:

• - Near-Zero Boot Time: Boots in milliseconds (just launching a sandboxed process, not booting an entire OS kernel).

• - Featherweight Footprint: Shares host kernel; container images often < 100 MB vs 20+ GB for a VM disk image.

• - Portability: Docker packages agricultural AI scripts with exact Python dependencies for uniform execution.

Study prompt: Explain this idea in your own words and connect it to an example from your discipline.

# Linux Storage Management: Logical Volume Manager (LVM)

## Dynamic Partitioning and Online Storage Resizing

• • The Static Partition Limitation: Traditional MBR/GPT partitions have fixed sizes; resizing requires unmounting filesystems and risks data loss.

• • LVM Architectural Layers:

• - Physical Volumes (PV): Raw physical partitions or entire disks initialized for LVM (`pvcreate /dev/sdb1`).

• - Volume Groups (VG): Storage pools created by aggregating multiple PVs into a unified storage pool (`vgcreate data_pool /dev/sdb1 /dev/sdc1`).

• - Logical Volumes (LV): Virtual partitions carved out of the Volume Group, formatted with a filesystem (ext4/XFS) and mounted (`lvcreate -L 50G -n crop_docs data_pool`).

• • Key Benefits: Online volume expansion (`lvextend -L +20G /dev/data_pool/crop_docs -r`) and atomic read-only storage snapshots for backup.

Study prompt: Explain this idea in your own words and connect it to an example from your discipline.

# Shell I/O Redirection & Pipeline Diagnostics

## Composing Modular Terminal Tools via Standard Streams

- • The Three Standard POSIX Streams:
- - Standard Input (`stdin`, File Descriptor `0`): Keyboard input by default.
- - Standard Output (`stdout`, File Descriptor `1`): Terminal screen display.
- - Standard Error (`stderr`, File Descriptor `2`): Unbuffered diagnostic error messages.
- • Redirection Operators:
- - `command > file.txt`: Redirects `stdout`, overwriting file.
- - `command >> file.txt`: Redirects `stdout`, appending to file.
- - `command 2> errors.log`: Redirects only `stderr` to file.
- - `command &> all_output.log`: Redirects both `stdout` and `stderr`.
- • Pipelines (`|`): Passes `stdout` of left command directly as `stdin` of right command:
- - Example: `ps aux | grep python3 | awk '{print $2, $4}' | sort -nr`.

Study prompt: Explain this idea in your own words and connect it to an example from your discipline.

# Topic 4: Key Takeaways & Synthesis Matrix

## Summary of Operating Systems Principles for Academic Mastery

• • OS Foundations: Resource Arbiter and Extended Machine; Kernel Mode (Ring 0) protects hardware from User Mode (Ring 3).

• • Kernels: Monolithic (fast, single address space) vs Microkernel (modular, IPC-based, isolated drivers).

• • Process Scheduling: Managed through process information; algorithms differ in order, fairness, and responsiveness.

• • Memory & Storage: Paging enables virtual memory via MMU/TLB; Journaling (NTFS/ext4) prevents metadata corruption during sudden power cuts.

• • MS-DOS: Driven by `IO.SYS`, `MSDOS.SYS`, and `COMMAND.COM`; internal commands resident in RAM, external commands on disk.

• • Systems Administration: Windows leverages PowerShell object pipelines; Linux leverages Bash text streams and FHS hierarchy.

Study prompt: Explain this idea in your own words and connect it to an example from your discipline.

# Review Quiz & Knowledge Check

## Self-Testing Key OS Concepts

• 1. Why is an operating system's kernel executed in Ring 0 while user web browsers execute in Ring 3?

• 2. What is the fundamental difference between an Internal DOS command and an External DOS executable?

• 3. Explain how a Journaling file system (such as ext4 or NTFS) prevents volume corruption when power fails.

• 4. What causes process 'Starvation' in Shortest Remaining Time First (SRTF) CPU scheduling?

• 5. Why is a standard desktop Linux or Windows OS unsuitable for hard real-time tractor guidance controls?

Study prompt: Explain this idea in your own words and connect it to an example from your discipline.

# Reading Assignments & Homework Notice

## Preparation for Next Lecture & Continuous Assessment

• • Required Readings:

• - Silberschatz, A., Galvin, P. B., & Gagne, G. - Operating System Concepts (10th ed.). Wiley. https://www.wiley.com/en-ie/Operating+System+Concepts%2C+10th+Edition-p-9781119320913

• - Smith, J. L., Wang, S., & Mortati, J. - Information Systems for Business and Beyond (2019), Chapter 3: Software. https://opentextbook.site/informationsystems2019/

• • Homework Assignment 4 Notice:

• - Available on SUA Moodle / Assignment directory: `BIT_111_Assignment_4.pdf`.

• - Total Allocation: 100 Marks (Covers Process Scheduling, Batch Scripting, and Administrative Utilities).

• - Submission Deadline: End of Week 8 by 17:00 EAT.

• • Next Lecture Preview: Topic 5 — Microcomputer Applications (Word Processing, Simple Spreadsheets, Internet Protocols).

Study prompt: Explain this idea in your own words and connect it to an example from your discipline.

# ONLINE FIGURE 1: Desktop Linux Mint 9 KDE.png

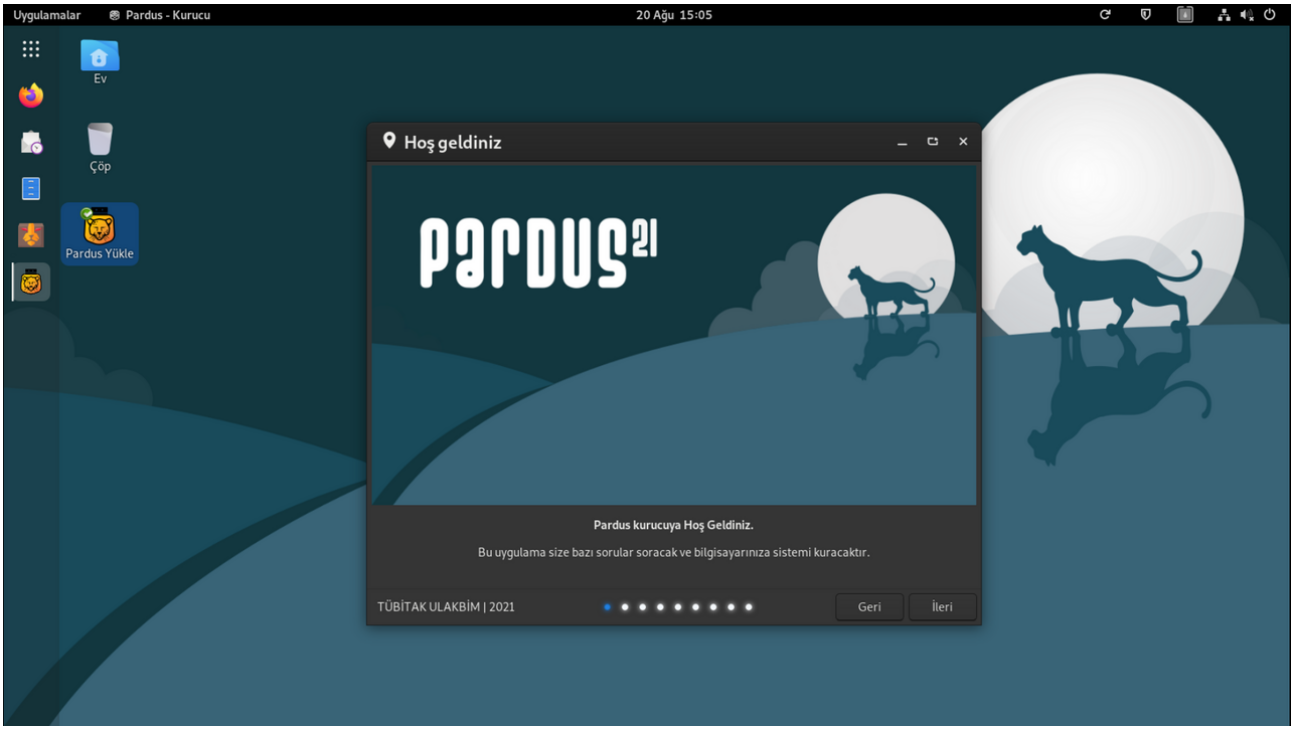



What to notice: Desktop Linux Mint 9 KDE

Topic 4 connection: Functions and Types of Operating Systems. Discuss what the figure helps you recognize and what information it does not provide by itself.

*Attribution: SteefuitHeerlen. Wikimedia Commons source page. License: CC BY 3.0.*

# ONLINE FIGURE 2: Ekran görüntüsü 2021-08-20 15-05-42.png




What to notice: Ekran görüntüsü 2021-08-20 15-05-42

Topic 4 connection: Functions and Types of Operating Systems. Discuss what the figure helps you recognize and what information it does not provide by itself.

*Attribution: AliIsingor. Wikimedia Commons source page. License: CC BY-SA 4.0.*

# ONLINE FIGURE 3: Linux-Mint-MATE-20-desktop.png

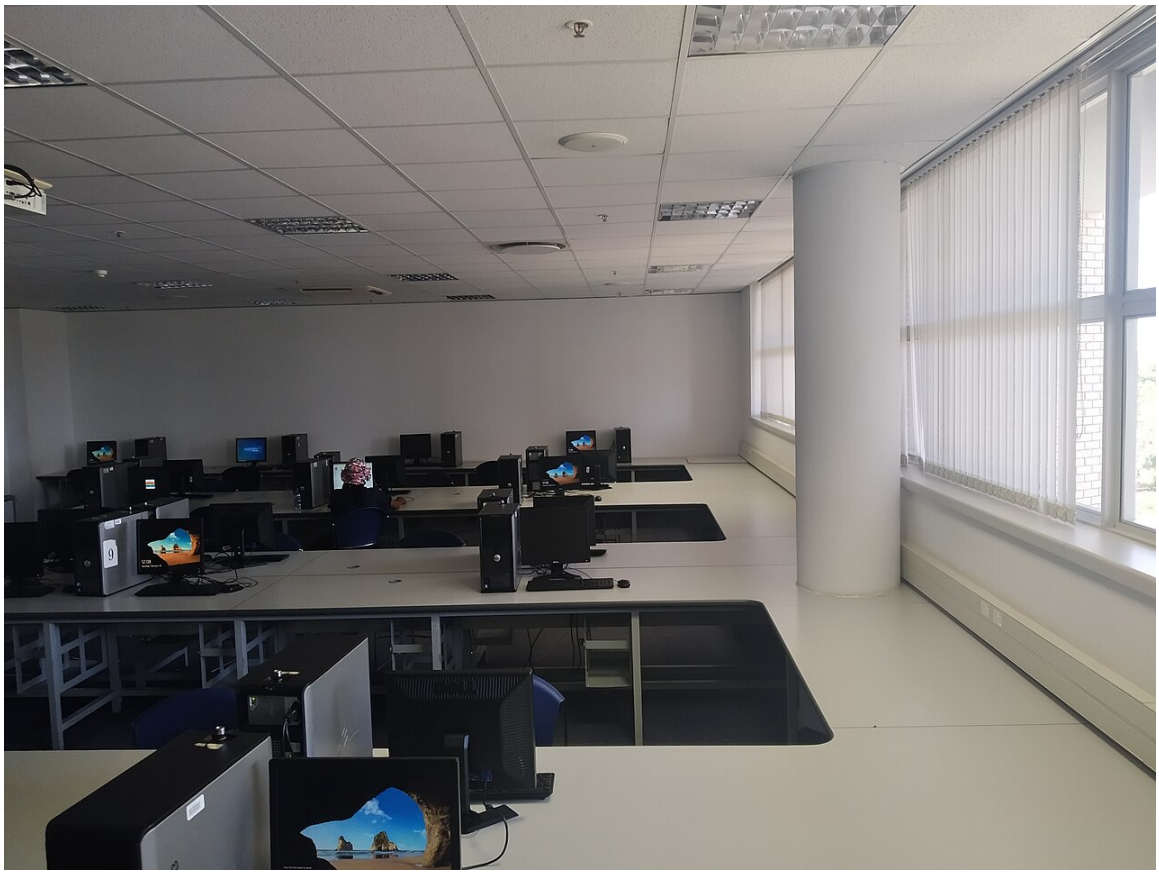



What to notice: Linux-Mint-MATE-20-desktop

Topic 4 connection: Functions and Types of Operating Systems. Discuss what the figure helps you recognize and what information it does not provide by itself.

*Attribution: Michal Stanke. Wikimedia Commons source page. License: CC BY-SA 4.0.*

# ONLINE FIGURE 4: Computer Science lab, at block 247 lab 293 at the University of Botswana.jpg



What to notice: Computer Science lab, at block 247 lab 293 at the University of Botswana

Topic 4 connection: Functions and Types of Operating Systems. Discuss what the figure helps you recognize and what information it does not provide by itself.

*Attribution: Sikolethu. Wikimedia Commons source page. License: CC BY-SA 4.0.*

# ONLINE FIGURE 5: De La Salle University – Dasmariñas (DLSU–D) Information and Communications Technology Center (ICTC) laboratory room 203 (ICT203) — normal view.jpg



What to notice: De La Salle University – Dasmariñas (DLSU–D) Information and Communications Technology Center (ICTC) laboratory room 203 (ICT203) — normal view

Topic 4 connection: Functions and Types of Operating Systems. Discuss what the figure helps you recognize and what information it does not provide by itself.

*Attribution: UndueMarmot. Wikimedia Commons source page. License: CC0.*

# ONLINE FIGURE 6: Computer lab showing desktop PCs warwick.jpg

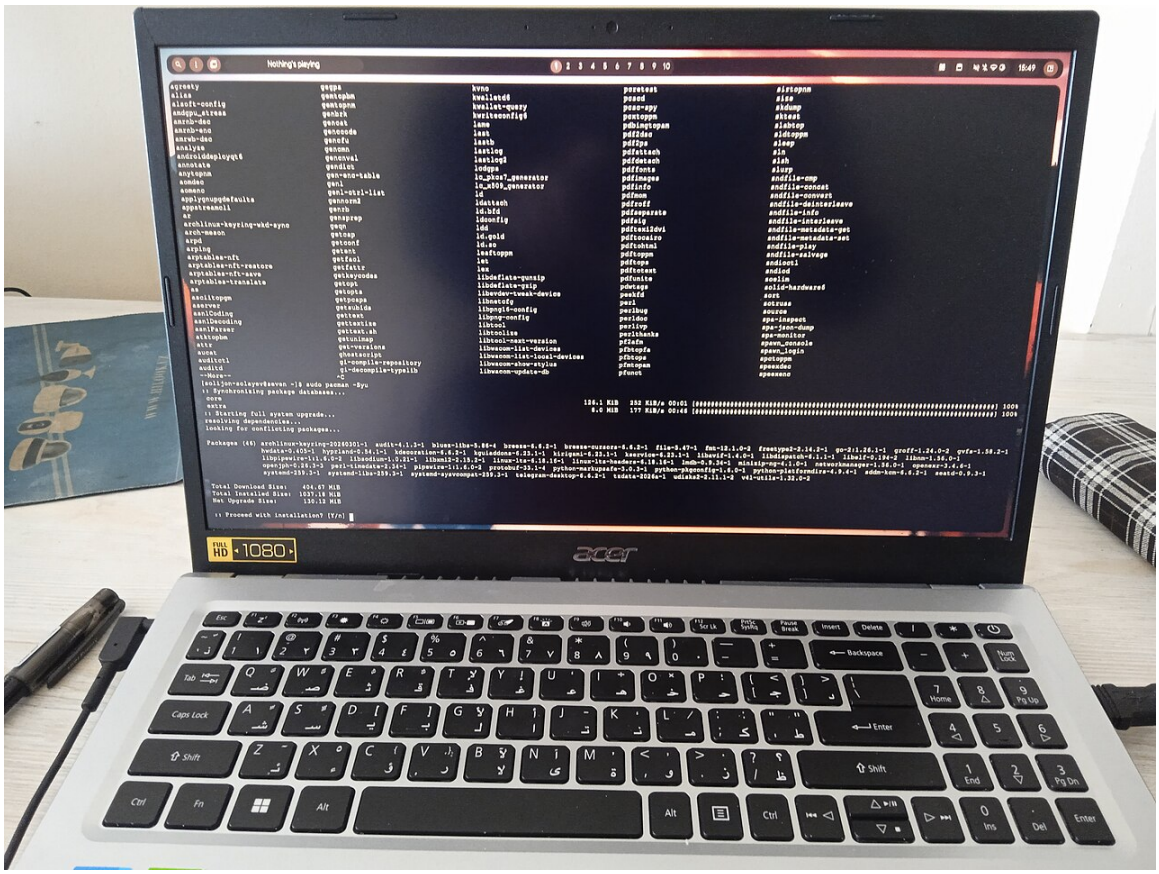


What to notice: Computer lab showing desktop PCs warwick

Topic 4 connection: Functions and Types of Operating Systems. Discuss what the figure helps you recognize and what information it does not provide by itself.

*Attribution: User Mike1024 on English Wikipedia. Wikimedia Commons source page. License: Public domain.*

# ONLINE FIGURE 7: Arch Linux system update via pacman on an Acer laptop.jpg




What to notice: Arch Linux system update via pacman on an Acer laptop

Topic 4 connection: Functions and Types of Operating Systems. Discuss what the figure helps you recognize and what information it does not provide by itself.

*Attribution: Solijon Solayev. Wikimedia Commons source page. License: CC BY-SA 4.0.*

## ONLINE FIGURE 8: Desktop computer system diagram (English).svg




What to notice: Desktop computer system diagram (English)

Topic 4 connection: Functions and Types of Operating Systems. Discuss what the figure helps you recognize and what information it does not provide by itself.

*Attribution: Photograph: kjarrett on Flickr. Wikimedia Commons source page. License: CC BY 2.0.*

# READING AND IMAGE CREDITS

Smith, J. L., Wang, S., & Mortati, J. (2019). Information Systems for Business and Beyond. Open textbook. https://opentextbook.site/informationsystems2019/

Vermaat, M. E., Sebok, S. L., Freund, S. M., Campbell, J. T., & Frydenberg, M. (2018). Discovering Computers: Digital Technology, Data, and Devices (16th ed.). Cengage. https://faculty.cengage.com/works/9781337285100

Online image credits appear beside each image. Images were selected from Wikimedia Commons and included only where a public-domain or stated reuse license could be checked on the file page.